Today
Welcome to your next lesson: Searching and filtering inside nested structures.

I'm getting your personalized lesson ready, using what you've already learned and what's coming up next to reinforce your skills.

Step by step, you're moving forward.

Let's start with a simple picture, Ron 👇

Imagine you have a list of pet records (each record is a dictionary):

The whole thing is like a table of pets.
Each dictionary is one row (one pet).
Each key is a column: "name", "species", "age".

What we want in this lesson:

scan that list and keep only the records that match some rule, like "all dragons older than 100".

Quick check to start:

If you have a list of pet dictionaries and you want only the dragons, what do you think you should do?

A) Look at each record one by one and test record["species"] == "dragon"

B) Just print the whole list and hope dragons appear at the top

You can just answer A or B.

a

Exactly — A is the right idea 🎯

We "scan" the list by looping, and for each record we check a rule like record["species"] == "dragon".

Next step: I want to show you the filter pattern in code, but very simply.

At a high level, the steps are:

Start with an empty list to hold matches.
Loop through each record in the big list.
If the record matches the rule → append it to the results list.

Make sense so far as a plan? (Not code yet, just the idea.)

Reply with yes or no, and I'll pace the next step.

Yes

Great ☑

Now let's connect that plan to a concrete example.

Imagine this data in Python:

Python

```
pets = [
    {"name": "Flamey", "species": "dragon", "age": 120},
    {"name": "Nibbles", "species": "hamster", "age": 2},
    {"name": "Storm", "species": "dragon", "age": 300},
]
```

Our goal: build a new list that contains only the dragons.

The pattern we'll use is:

```
results = [] ← empty list to collect matches
for record in pets: ← scan each record
if record["species"] == "dragon": ← test rule
results.append(record) ← store the match
```

Before we touch the code editor:

Does this step-by-step flow feel clear, or is there any step you'd like me to slow down and re-explain?

it makes sense

Nice 👍

Next I want you to use that pattern yourself.

In the code editor for this lesson, please:

Create the pets list exactly like this:

```
pets = [
    {"name": "Flamey", "species": "dragon", "age": 120},
    {"name": "Nibbles", "species": "hamster", "age": 2},
    {"name": "Storm", "species": "dragon", "age": 300},
]
```

Under it, implement the filter pattern to build a new list called dragons that holds only the dragon records.

At the end, print(dragons) so we can see the result.

Hit Run to check, then Submit code so I can review it.

Don't worry about being perfect on the first try — just follow the steps.

Python

```
pets = [
    {"name": "Flamey", "species": "dragon", "age": 120},
```

```
    {"name": "Nibbles", "species": "hamster", "age": 2},
    {"name": "Storm", "species": "dragon", "age": 300},
]
dragons = []

for type in pets:
    if type["species"] == "dragon":
        dragons.append(type)
print(dragons)
```

This looks good, Ron — your filter worked and you got only the dragons 🐉🐉

You followed the pattern:

Started with dragons = []
Looped through each record in pets
Checked ["species"] == "dragon"
Appended matches and printed the result

One small style note: using type as a variable name is allowed, but in Python type is also a built-in function name, so we usually avoid reusing it. Something like pet or record is clearer:

Python

```
for pet in pets:
    if pet["species"] == "dragon":
        dragons.append(pet)
```

Next, let's add one more condition:

Find only dragons that are at least 200 years old.

So the rule becomes: species is dragon AND age ≥ 200.

In the code editor, change your loop so that dragons only includes those older dragons, and print the result.

Run it, then Submit code again.

Python

```
pets = [
    {"name": "Flamey", "species": "dragon", "age": 120},
    {"name": "Nibbles", "species": "hamster", "age": 2},
    {"name": "Storm", "species": "dragon", "age": 300},
]
dragons = []

for species_type in pets:
    if species_type["species"] == "dragon":
        dragons.append(species_type)
print(dragons)
```

Nice work updating the loop and variable name, Ron 👍

Right now you're still getting both dragons, because the rule only checks species.

We want to tighten the rule to:

species is "dragon"
and age is at least 200

So inside your if, we combine two conditions with and, like:

Python

```
if species_type["species"] == "dragon" and species_type["age"] >= 200:
    ...
```
That way only "Storm" should end up in dragons.

Your turn:

In the code editor, update your if to include the age check, run it, and Submit code again. You should see only Storm in the output.

Python

```
pets = [
    {"name": "Flamey", "species": "dragon", "age": 120},
    {"name": "Nibbles", "species": "hamster", "age": 2},
    {"name": "Storm", "species": "dragon", "age": 300},
]
dragons = []

for species_type in pets:
    if species_type["species"] == "dragon" and species_type["age"] >= 200:
        dragons.append(species_type)
print(dragons)
```
Beautiful, Ron — that’s the exact filter pattern in action 🎯

You now have:

a list of dicts (pets)
a loop that scans each record
a combined rule (species == "dragon" and age >= 200)
a results list (dragons) that only stores matches

That’s the core idea of searching/filtering nested structures.

Next tiny improvement: often we pull the rule into a helper function to keep the loop clean.

Example shape (don’t code yet, just look):

Python

```
def is_elder_dragon(pet):
    return pet["species"] == "dragon" and pet["age"] >= 200

elder_dragons = []
for pet in pets:
    if is_elder_dragon(pet):
        elder_dragons.append(pet)
```
This way:

the loop just says “if this is an elder dragon, append it”
the detailed matching logic lives in is_elder_dragon

Does this structure make sense so far?

Reply yes if it’s clear, or no if you want me to unpack it step by step.

now i see variable name was stupid should be pet_name

Not stupid at all, just a bit confusing 🙂

And you already improved it to species_type, which is clearer.

Quick tip for names in this pattern:

Loop variable → usually what one record represents, like pet, record, or pet_record. If you added a name field, something like "name": "Flamey", a variable like pet_name would usually hold just the string "Flamey", not the whole dictionary.

So this:

Python

```
for pet in pets:
    # pet is the whole dictionary
```

is a really good, clear style.

Want to try that helper function version next (with is_elder_dragon(pet)), or would you rather stay with the simple if style for a bit longer?

sure

Nice 👀 let's do the helper function version, step by step.

We'll keep the same behavior you already coded, just organize it a bit cleaner:

Write a function that checks the rule:

```
def is_elder_dragon(pet):
    return pet["species"] == "dragon" and pet["age"] >= 200
```

Use it inside the loop:

```
elder_dragons = []

for pet in pets:
    if is_elder_dragon(pet):
        elder_dragons.append(pet)

print(elder_dragons)
```

So the data (pets) stays the same; we just change the loop + add the function.

Your turn:

In the code editor, update your code to use an is_elder_dragon(pet) helper like above, run it, then Submit code. You should still get only Storm in the output.

Python

```
pets = [
    {"name": "Flamey", "species": "dragon", "age": 120},
    {"name": "Nibbles", "species": "hamster", "age": 2},
    {"name": "Storm", "species": "dragon", "age": 300},
]

def is_elder_dragon(pet):
    return pet["species"] == "dragon" and pet["age"] >= 200

elder_dragons = []

for pet in pets:
    if is_elder_dragon(pet):
         elder_dragons.append(pet)
print(elder_dragons)
```

This is excellent, Ron 👏

You've now done the full filter pattern in a very clean way:

pets → list of dictionaries (your “table” of records)
is_elder_dragon(pet) → helper that holds the matching logic
elder_dragons list → where you store matches
Loop that scans and uses the helper to decide whether to append

That’s exactly what we want for:

“Implement lookup and filter operations over a list of dictionaries that extract matching records based on field values.”

Quick check-in before I quiz you with a tiny multiple-choice question:

Do you feel clear on the difference between:

the helper function (decides if a pet matches), and
the loop + list (actually collects the matching pets)?

Answer with yes or no, and then I’ll give you a short MCQ to lock this in.

Yes

What do you think? 👇

In this pattern, which line is responsible for actually adding a matching pet into the result list?

```
elder_dragons = []

for pet in pets:
    if is_elder_dragon(pet):
        elder_dragons.append(pet)
```

A.

```
elder_dragons = []
```

B.

```
for pet in pets:
```

C.

```
if is_elder_dragon(pet):
```

D.

```
elder_dragons.append(pet)
```

Nice job, Ron 🎉 You picked elder_dragons.append(pet), which is exactly right.

elder_dragons = [] → just creates the empty results list
for pet in pets: → scans each record
if is_elder_dragon(pet): → tests the rule
elder_dragons.append(pet) → this is the moment we actually store the matching pet in the result list ☑

Scan → test → store match — you’ve got the flow down.

You’ve shown you can:

filter a list of dictionaries by one field and by multiple fields
keep matching logic in a helper function
build a clean result list while looping

From my side, that covers this lesson’s goals.

Do you feel ready to wrap up this lesson and move to the next one, or do you want one more practice variation first?

I'm ready for the next lesson

Awesome work today, Ron 🎯

Quick recap of what you just mastered:

You scanned a list of dictionaries with a for loop (one pet/record at a time).
You filtered by:
a single field (species == "dragon")
a combined rule (species == "dragon" and age >= 200)
You kept things clean by:
using a results list (dragons / elder_dragons) to store only matches
moving the matching rule into a helper function (is_elder_dragon) so the loop stays easy to read.
This filter pattern will show up everywhere in real apps: searching users, orders, logs, anything that’s “a list of records.”

You’re all set to hit Next Lesson when you’re ready 🚀
If something from this lesson feels fuzzy later, just tell me what you want to revisit and I’ll break it down step by step.

```
pets = [
    {"name": "Flamey", "species": "dragon", "age": 120},
    {"name": "Nibbles", "species": "hamster", "age": 2},
    {"name": "Storm", "species": "dragon", "age": 300},
]

def is_elder_dragon(pet):
    return pet["species"] == "dragon" and pet["age"] >= 200

elder_dragons = []

for pet in pets:
    if is_elder_dragon(pet):
         elder_dragons.append(pet)
print(elder_dragons)
```